Brief Announcement: Toward Uniform Content-Oblivious Leader Election on General Graphs

Fabian Frei[*†]
MIT
Cambridge, USA
ffrei@mit.edu

Ran Gelles
Bar-Ilan University
Ramat Gan, Israel
ran.gelles@biu.ac.il

Ahmed Ghazy
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
ahmed.ghazy@cispa.de

Alexandre Nolin[‡§]
Télécom SudParis
Palaiseau, France
alexandre.nolin@telecom-sudparis.eu

Abstract

In the content-oblivious model, communication is limited to sending content-less pulses over asynchronous channels. Despite this extreme restriction, Censor-Hillel et al. (Dist. Comp., 2023) showed that *any* computation can be simulated on 2-edge-connected graphs, assuming a designated leader. Subsequent work investigated the necessity of this assumption. Frei et al. (DISC 2024, Dist. Comp. 2026) and Chalopin et al. (DISC 2025) designed content-oblivious leader-election algorithms for rings, thereby eliminating the need for an initial leader. Non-uniform leader election is possible on 2-edge-connected graphs (Chalopin et al., DISC 2025).

We extend *uniform* leader election beyond rings to a significantly broader class of topologies. Specifically, we present a uniform content-oblivious leader-election algorithm for Eulerian graphs with message complexity $O(|E| \cdot \mathrm{ID}_{\max})$, where $|E|$ is the number of edges, and $\mathrm{ID}_{\max}$ is the largest identifier in the network. While the feasibility of uniform content-oblivious leader election in arbitrary 2-edge-connected graphs remains open, our result substantially narrows the gap between rings and general 2-edge-connected graphs.

CCS Concepts

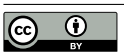
• **Theory of computation** → Distributed algorithms; Distributed computing models; • **Networks** → Network reliability.

Keywords

Leader Election, Eulerian Networks, Content-Oblivious Computation, Fully Defective Networks, OR Protocol

ACM Reference Format:
Fabian Frei, Ran Gelles, Ahmed Ghazy, and Alexandre Nolin. 2026. Brief Announcement: Toward Uniform Content-Oblivious Leader Election on General Graphs. In *ACM Symposium on Principles of Distributed Computing (PODC '26), July 6–10, 2026, Egham, United Kingdom.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3796701.3815949

[*]Part of this work was done at the CISPA Helmholtz Center for Information Security.
[†]Also with ETH Zurich.
[‡]Also with Inria.
[§]Also with Institut Polytechnique de Paris.



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '26, Egham, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2512-8/26/07
https://doi.org/10.1145/3796701.3815949

1 Introduction

Modern distributed systems often operate under extreme communication faults caused by noise, interference, and device malfunctions. Motivated by this challenge, Censor-Hillel et al. [2] introduced the *content-oblivious* model, an asynchronous message-passing model in which the content of every message may be adversarially altered. In this setting, nodes cannot rely on message payloads and may use only the existence and arrival order of content-less messages (*pulses*) to coordinate computation.

Censor-Hillel et al. [2] showed content-oblivious computation to be far more powerful than initially expected. They proved that in any 2-edge-connected network with a designated leader, any asynchronous distributed algorithm can be simulated in a content-oblivious manner. At the same time, they established a matching impossibility result: non-trivial computation is impossible in networks that are not 2-edge-connected. Together, these results positioned 2-edge-connectivity as the structural property that enables arbitrary computations. However, the question of whether the assumption of a designated leader is necessary remains open.

Removing this assumption reduces to designing a leader-election algorithm in the content-oblivious model. The end of the algorithm should be clear to the processes and leave no lingering pulses in the network, so as not to pollute the subsequent execution of another algorithm. This was first achieved for ring topologies by Frei et al. [6, 7], who presented a deterministic leader-election algorithm for oriented rings with message complexity $O(n \cdot \mathrm{ID}_{\max})$, where n is the size of the ring and $\mathrm{ID}_{\max}$ denotes the maximum identifier assigned to a node. Chalopin et al. [4] extended this result to unoriented rings with the same asymptotic complexity. Importantly, these algorithms are *uniform*: nodes know neither n nor a bound on it. For non-uniform algorithms, Chalopin et al. [4] established that leader election is possible in *any* 2-edge-connected graph in the content-oblivious model, provided that the nodes know an upper bound $N \geq n$ on the number of nodes. Whether *uniform* leader election can be extended beyond rings has remained open, however.

Our results. We give a uniform leader election algorithm for *Eulerian* networks, i.e., whose nodes all have even degree.

Theorem 1. *Let $G = (V, E)$ be an Eulerian network where nodes are assigned unique identifiers. There is a deterministic uniform content-oblivious leader-election algorithm using $O(|E| \cdot \mathrm{ID}_{\max})$ pulses.*

Challenges. Extending uniform content-oblivious leader election beyond rings presents several obstacles. First, rings form a closed path. A message sent in one direction (i.e., clockwise or counterclockwise) and forwarded by all nodes traverses the entire ring,

eventually returning to the sender from the opposite side. This property holds even if nodes do not share a consistent view of which direction is clockwise and which is counterclockwise. In general graphs, however, nodes completely lack a sense of directionality and pulses' paths cannot be guaranteed. Nevertheless, the approach in [4] cleverly bypasses this issue by sending pulses *in all directions* in a synchronized manner. Each node sends one pulse through each of its ports, and then waits to receive one pulse from each port. This establishes a synchronized counting mechanism in which nodes progress in rounds of sending and receiving a pulse from all neighbors, analogous to an α-synchronizer [1].

Synchronized counting alone is insufficient. Neighboring nodes can convey only very limited information to one another if they exchange merely directed pulses. In particular, how can a node distinguish a counting pulse from a pulse intended to carry a different meaning? When nodes share knowledge of an upper bound N on the number of nodes, they can use it to distinguish these cases: sending N pulses along a single edge can signal the end of the counting phase [4]. It is unclear, however, how to resolve this ambiguity without this additional knowledge.

An initial attempt. Our approach leverages the structure of the network. It is well known that any Eulerian graph can be decomposed into edge-disjoint circuits [10]. Let v be a node of even degree $d = \deg(v)$. We conceptually split v into $d/2$ *virtual* processes, each managing two ports. Concretely, we fix an arbitrary pairing $(1,2),(3,4),\dots,(d-1,d)$ of v's incident ports into $d/2$ disjoint pairs. For each $j \in \{1,\dots,d/2\}$, we run an independent instance of the ring leader election [4] using only the two ports $(2j-1, 2j)$ at v, which we equivalently treat as a virtual node $v^{(j)}$ incident solely to the corresponding two edges. Since every port belongs to exactly one pair and each virtual node $v^{(j)}$ has degree exactly 2, this local wiring induces a collection of edge-disjoint simple cycles (i.e., rings) [10, p. 16]. Hence, each $v^{(j)}$ belongs to exactly one ring and participates in a single instance of leader election on that ring.

Assigning to each $v^{(j)}$ the identifier $\mathsf{ID}_{v^{(j)}} = \mathsf{ID}_v \circ j$, using a prefix-free encoding, ensures that all virtual nodes have unique identifiers. It follows that on every induced ring, the corresponding ring leader election instance elects a *unique* leader.

From this point, the computation is straightforward. Since the conditions required by the general compiler [2] hold, each ring can establish reliable communication independently. The leaders of the individual rings can then execute a (standard) asynchronous message-passing leader election among themselves to obtain a network-wide leader. Communication within a ring is implemented via the general compiler [2], while communication between rings is realized through direct information exchange between the different virtual instances of the same physical node.

Complexity and Optimizations. While capturing the essence of our construction, an algorithm directly following the above overview incurs a much higher message complexity than the one of Theorem 1. Two optimizations allow us to reduce the complexity.

First, assigning virtual identifiers of the form $\mathsf{ID}_v \circ j$ ensures virtual nodes have unique identifiers, but can result in the maximum virtual identifier being as large as $\Theta(n \cdot \mathsf{ID}_{\max})$, increasing the cost of running leader election on each ring.

To keep the dependence only on $\mathsf{ID}_{\max}$, we instead let *all* virtual instances of a node v use the same identifier ID_v. This creates two collisions. First, a single circuit may visit the same node multiple times, hence multiple virtual nodes on that circuit share the same identifier. In the general case this may compromise the correctness of the underlying ring leader-election algorithm, but in our case, these virtual nodes are all controlled by the same physical node and can coordinate. Specifically, node v designates one instance as the *active* participant in the ring algorithm, while all other instances behave as pure relays (forwarding pulses between their two incident edges). Second, distinct circuits that pass through the same physical node v may each elect v as their leader. This complicates the next step, in which circuits need unique identifiers to coordinate with one another. However, when v detects that it is elected leader in multiple incident circuits, it can resolve this situation *locally*. Before moving on to the next step, node v re-pairs its incident ports so that all such circuits are merged into a single induced circuit.

Second, using an existing leader-election algorithm as a black box to elect a leader between circuits (through [2]) comes with a high simulation overhead. We present a more efficient approach based on *circuit merging*. In the spirit of the classical GHS algorithm [8], each circuit selects the neighboring circuit whose leader has the highest identifier among the adjacent circuits and merges into it. The merge occurs by a node incident to both circuits locally re-pairing its ports so that the two circuits become segments of a single circuit. The maximum-ID node on the merged circuit becomes its new leader. Repeating this merging step a logarithmic number of times yields a single Eulerian circuit and a unique network-wide leader. In total, this process uses at most $O(|E| \cdot \log \mathsf{ID}_{\max} \cdot \log n)$ pulses and does not increase the overall asymptotic complexity.

2 Uniform Leader-Election Algorithm for Eulerian Networks

Our algorithm consists of four steps: first, it splits the network into disjoint circuits (Lemma 2); second, it elects a leader on each (Lemma 3); third, it merges circuits with the same leader (Lemma 4); finally, it merges all the circuits into a single circuit, and outputs the leader of that circuit as global leader (Lemma 5).

2.1 Circuit Decomposition and Electing a Leader in Each Circuit

To set the stage for the rest of the algorithm, we require a *circuit decomposition* of the network, that is, a partition of its edges into circuits, on which a ring leader-election algorithm [4] can be executed. It is well known that a graph admits a circuit decomposition if and only if it is Eulerian [9, Lemma 1].

A circuit decomposition of the network induces a *port pairing*, i.e., a partition of each node's ports into pairs: the two consecutive edges (u,v) and (v,w) of a circuit pair the corresponding ports of v. Although such a pairing is not given to the nodes a priori, we can use an arbitrary pairing since the converse also holds in Eulerian graphs. Namely, every pairing of the edges incident to every node induces a unique circuit decomposition of the graph [9, Lemma 4].

Lemma 2. *A circuit decomposition of the network can be computed without any communication between nodes by fixing, for any node v, the port pairing* $(1,2),\dots,(\deg(v)-1,\deg(v))$.

The next step is to elect a leader in each circuit by use of the ring leader-election algorithm [4], denoted by $\mathcal{A}_{\text{LE}}(\text{ID}_v, i, j)$ for a node v with identifier ID_v and ports i and j. Recall that circuits, unlike cycles, may revisit vertices; neither uses an edge twice. As explained in the introduction, there are two challenges in our setting. First, while $\mathcal{A}_{\text{LE}}$ is available, it operates for rings (simple cycles) rather than arbitrary circuits. Second, it requires distinct identifiers, which is not the case when a circuit goes through the same node several times. The first challenge is addressed by leveraging the port pairings computed in the first step; each node v is split into $\deg(v)/2$ virtual nodes of degree 2 based on its pairing, effectively turning every circuit into a ring. We deal with the second challenge by having exactly one virtual node participate in $\mathcal{A}_{\text{LE}}$ while all other virtual nodes act as links and only passively relay pulses.

Lemma 3. *Given a circuit decomposition of the network, electing a leader on every circuit takes $O(|E| \cdot \text{ID}_{\max})$ pulses.*

Proof Sketch. Every node v splits itself into $\deg(v)/2$ virtual nodes $v^{(1)}, \dots, v^{(\deg(v)/2)}$ and assigns each $v^{(j)}$ a different port pair according to the circuit-decomposition-inducing port pairing guaranteed by Lemma 2. Every node v arbitrarily chooses a virtual node $v^{(j)}$ as its *active* node. We call the other virtual nodes *passive*. The active node $v^{(j)}$ then starts executing the given ring leader-election algorithm using the identifier of v, while the passive nodes act as relays between their assigned ports and log what is relayed.

On each circuit of the given circuit decomposition, we may now have zero, one, or several active virtual nodes. In any circuit without active nodes, there will be no pulses at this stage. In any circuit with at least one active virtual node, a leader is elected. As soon as a leader is elected in the circuit of an active node $v^{(j)}$, the node v checks whether any of its virtual nodes has not seen any pulses yet. In this case, it arbitrarily picks such a virtual node as its next active node, i.e., it starts executing ring leader election on this virtual node, again with the identifier of v. This continues for v until each of its virtual nodes has run a ring leader-election instance as an active node or relayed at least one pulse as a passive node. Then v continues to wait until every passive node has observed the pulse pattern of a complete ring leader-election algorithm. Once this has happened, an instance of the ring leader-election algorithm was executed on each circuit passing through v.

We note that, crucially, for every instance of the ring leader-election algorithm, the involved virtual nodes have unique identifiers as required, as for every physical node at most one virtual node is active at a time. Executing $\mathcal{A}_{\text{LE}}$ on a cycle C_i sends $O(|C_i| \cdot \text{ID}_{\max})$ pulses [4]. This step sends $O(|E| \cdot \text{ID}_{\max})$ pulses overall. □

The above lemma guarantees that a leader is eventually elected on each circuit. However, it is possible that the same node is elected leader in multiple circuits. We address this issue next.

Lemma 4. *For any node v elected leader in multiple incident circuits, node v can* locally *merge them into a single circuit in which v is the leader without any communication.*

Proof Sketch. Let $(a_1, b_1), \dots, (a_k, b_k)$ be, w.l.o.g., the port pairing of v with (a_i, b_i) belonging to a circuit C_i in which v became the leader by running $\mathcal{A}_{\text{LE}}(\text{ID}_v, a_i, b_i)$. Node v re-labels its port pairings to $(b_1, a_2), (b_2, a_3), \dots, (b_k, a_1)$. Note that v is the only node in $C_1, \dots, C_k$ that changes its pairings. This new pairing induces a single circuit that is a concatenation of circuits $C_1, \dots, C_k$. □

2.2 Merging Circuits to an Eulerian Circuit

In the final step of our algorithm, circuits progressively merge until a single Eulerian circuit remains. The merging follows the spirit of the classical GHS algorithm [8], where disjoint connected components merge to form a spanning tree. The process is also fully asynchronous, with merges occurring in synchronized *phases* orchestrated by the circuit leaders, and only circuits belonging to the same phase may merge. In each phase, every circuit C selects one neighboring circuit C' and merges into it. Importantly, during the same phase, C' may itself merge with another circuit C'', and so on. Thus, an entire chain of circuits may collapse into a single larger circuit within one phase, with a single leader chosen to be the "last" one of this chain. A single Eulerian circuit with a single leader eventually remains, after at most $O(\log n)$ phases.

Lemma 5. *There is an algorithm using $O(|E| \cdot \log \text{ID}_{\max} \cdot \log n)$ pulses merging given circuits, each with a leader, into a single circuit.*

Proof Sketch. We describe how a circuit C behaves in phase k. In $O(|C| \cdot \log \text{ID}_{\max})$ pulses nodes in C agree on an orientation of C and learn the identifier of their leader ℓ_C. ℓ_C checks whether C is adjacent to any other circuit C'. As two edge-disjoint circuits C and C' have distinct leaders, nodes at the intersection of C and C' can tell the two circuits apart. If no adjacent C' exists, C is Eulerian, and the algorithm is over. The leader learns this by executing an OR protocol where every node inputs 1 if it knows about an adjacent circuit and 0 otherwise. In the OR protocol, ℓ_C initiates a clockwise pulse. The first node with input 1 reflects it; otherwise, if all inputs are 0, the pulse completes the cycle. In both cases, the leader broadcasts the outcome by forwarding the pulse along the cycle. If all inputs are 0, every node observes two clockwise pulses. Otherwise, nodes either observe the reflection or receive a counterclockwise pulse without a clockwise one; both indicate output 1 and allow termination. A similar OR protocol appeared in a recent concurrent work [5].

Next, each circuit C computes the largest identifier among its adjacent circuits. This is done in $O(|C| \cdot \log \text{ID}_{\max})$ pulses. Each node on C belonging to multiple circuits selects the circuit C' among them whose leader $\ell_{C'}$ has the maximal identifier and encodes this identifier in prefix-free form. Subsequently, all nodes in C participate in a sequence of OR protocols applied to the bits of these encodings so that the presence of a higher identifier is detected whenever some node contributes it (causing the other nodes to yield in subsequent OR instances), until the leader obtains all bits of the maximal identifier provided by the participating nodes. A similar approach appeared in [3, Thm. 3].

Finally, a unique designated node in C that borders the maximal C' (e.g., the first clockwise one from the leader) merges the two circuits by locally chaining its port pairing; this step is done in a synchronized way, namely, when both C and C' have completed all other tasks of phase k. □

Acknowledgments

This work of Fabian Frei was supported in part by the Swiss National Science Foundation (SNSF) under Grant No. 218001.

References

[1] Baruch Awerbuch. 1985. Complexity of Network Synchronization. *J. ACM* 32, 4 (Oct. 1985), 804–823.

[2] Keren Censor-Hillel, Shir Cohen, Ran Gelles, and Gal Sela. 2023. Distributed computations in fully-defective networks. *Distributed Computing* 36, 4 (2023), 501–528.

[3] Jérémie Chalopin, Yi-Jun Chang, Lyuting Chen, Giuseppe A. Di Luna, and Haoran Zhou. 2025. Brief Announcement: Non-Uniform Content-Oblivious Leader Election on Oriented Asynchronous Rings. In *Proc. DISC (LIPIcs)*. 51:1–51:7.

[4] Jérémie Chalopin, Yi-Jun Chang, Lyuting Chen, Giuseppe A. Di Luna, and Haoran Zhou. 2025. Content-Oblivious Leader Election in 2-Edge-Connected Networks. In *Proc. DISC (LIPIcs, Vol. 356)*. 21:1–21:22.

[5] Jérémie Chalopin, Yi-Jun Chang, Giuseppe A. Di Luna, and Haoran Zhou. 2026. Efficient Counting and Simulation in Content-Oblivious Rings. In *PODC'26*.

[6] Fabian Frei, Ran Gelles, Ahmed Ghazy, and Alexandre Nolin. 2024. Content-Oblivious Leader Election on Rings. In *Proc. DISC (LIPIcs, Vol. 319)*. 26:1–26:20.

[7] Fabian Frei, Ran Gelles, Ahmed Ghazy, and Alexandre Nolin. 2026. Content-oblivious leader election on rings. *Distributed Computing* 39, 2 (April 2026), 19 pages. doi:10.1007/s00446-025-00491-x

[8] R. G. Gallager, P. A. Humblet, and P. M. Spira. 1983. A Distributed Algorithm for Minimum-Weight Spanning Trees. *ACM Trans. Program. Lang. Syst.* 5, 1 (Jan. 1983), 66–77.

[9] Anton Kotzig. 1968. Moves without forbidden transitions in a graph. *Matematický časopis 18.1* 18, 1 (1968), 76–80.

[10] Oswald Veblen. 1922. *Analysis Situs*. American Mathematical Society, New York. 150 pages.